

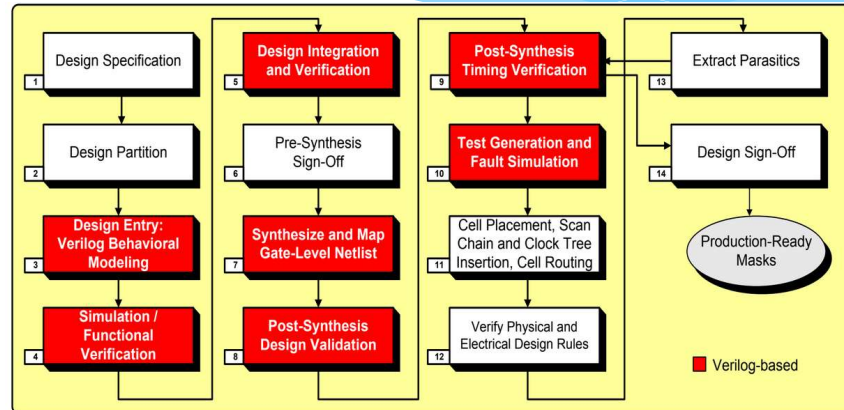
Introduction to Hardware Description Language

Kausik Datta
Mentor Graphics India Pvt. Ltd.
Kausik_Datta@mentor.com

What is Electronic Design

- * Developing a circuit by connecting **known electronic devices** to meet the requirement of a given **specification**
- * Specification
 - * Description of the desired behavior of circuit
- * Known electronic device
 - * Behavior can be modeled by known equations or algorithms, with known input values

Design Methodology



3

27 August 2024

Complexity

- * The electronic circuits which are designed nowadays are extremely complex
- * Designer can't handle the whole design as a single entity.
 - * The design has to be broken down into a set of smaller number of entities.
- * Design Approach:
 - * Modular Architecture
 - * Hierarchical
 - * Re-use of smaller blocks
 - * **Text based description (HDL)**

4

27 August 2024

Hardware Description Language

- * Programming Language to describe the Hardware at higher abstraction level – shorthand for describing digital hardware
- * Verify the behavior before implementation (Simulation)
- * Synthesize the description to implementation (behavioral to structural synthesis)

5

27 August 2024

Verilog

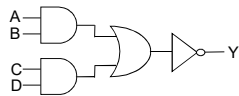
- * Gateway Design Automation started developing this language in 1985
- * Phil Moorby designed the basic language and the simulator
- * Cadence Design System acquired Gateway in 1989
- * Cadence opens Verilog in 1990
- * Easy to learn and master –
 - * syntactically similar to C
- * Can be used to model system in almost all stages of EDA flow
 - * Behavioural
 - * RTL (register-transfer level)
 - * Structural
 - * Analog – Verilog AMS
- * Enhanced and exists in different version
 - * Verilog 95
 - * Verilog 2001 (V2K)
 - * System Verilog (SV)

6

27 August 2024

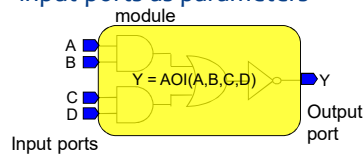
Structural design

- * AOI – AND OR INVERT



- * 4 gates to describe
 - * 2 AND gates
 - * 1 OR gate
 - * 1 INVERTER
- * Interconnection between gates

- * Verilog defines **module** as a hierarchical block
- * Module have input and output ports
- * Output ports are driven by some Boolean equations with input ports as parameters



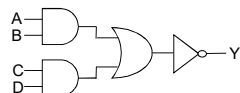
8

27 August 2024

Module definition



- * Verilog file AOI.v
- ```
// AOI module definition
module AOI;
```

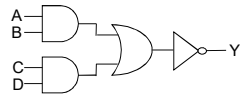


```
endmodule
```

9

27 August 2024

# Port declarations



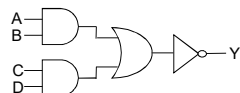
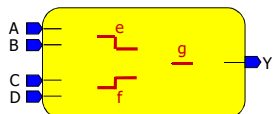
```
* Verilog file AOI.v
// AOI module definition
module AOI(Y, A, B, C, D);
// port declaration
input A, B, C, D;
output Y;
```

```
endmodule
```

10

27 August 2024

# Internal net declarations



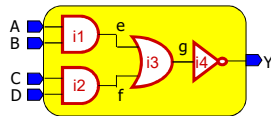
```
* Verilog file AOI.v
// AOI module definition
module AOI(Y, A, B, C, D);
// port declaration
input A, B, C, D;
output Y;
// internal wire declaration
wire e, f, g;
// gate instantiations
```

```
endmodule
```

11

27 August 2024

# Instantiations

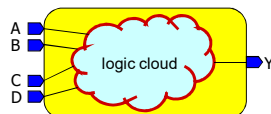


```
* Verilog file AOI.v
// AOI module definition
module AOI(Y, A, B, C, D);
// port declaration
input A, B, C, D;
output Y;
// internal wire declaration
wire e, f, g;
// gate instantiations
and i1(e, A, B);
and i2(f, C, D);
or i3(g, e, f);
not i4(Y, g);
endmodule
```

12

27 August 2024

# Behavioral representation



```
* Verilog file AOI.v
// AOI module definition
module AOI(Y, A, B, C, D);
// port declaration
input A, B, C, D;
output Y;
// internal wire declaration
wire e, f, g;
// boolean logic equation
assign e = A & B;
assign f = C & D;
assign g = e | f;
assign Y = ~g;
endmodule
```

13

27 August 2024

# Behavioral representation



```
* Verilog file AOI.v
// AOI module definition
module AOI(Y, A, B, C, D);
// port declaration
input A, B, C, D;
output Y;
// boolean logic equation
assign Y = ~((A & B) | (C & D));
endmodule
```

14

27 August 2024

# Learning

- \* The module starts with **module** keyword and finishes with **endmodule**
- \* Internal signals are named with **wire**
- \* Comments starts with //
- \* **input** and **output** ports are declared at the beginning of module
- \* Declarations, Statements and Instantiations end with a semicolon (;)
- \* Multiple declarations are separated with comma (,)

15

27 August 2024

# Basic Syntax of Module

module module\_name (list\_of\_ports);

input/output declarations

local net declarations

Concurrent statements

endmodule

```
* Verilog file AOI.v
// AOI module definition
module AOI(Y, A, B, C, D);
// port declaration
input A, B, C, D;
output Y;
// internal wire declaration
wire e, f, g;
// boolean logic equation
assign e = A & B;
assign f = C & D;
assign g = e | f;
assign Y = ~g;
endmodule
```

16

27 August 2024

# Variable Data Type

- \* Net (wire, wand, supply0 etc.)
  - \* Must be continuously driven
  - \* Used to model only combinational logic
  - \* Can be connected to input and output port of instance
  - \* The only legal type on LHS of assign statement
  - \* Multiple driver allowed
- \* Reg
  - \* Retains the last assigned value
  - \* Used to model both combinational and sequential logic
  - \* Can be connected to only input port of instance
  - \* The only legal type on LHS inside initial and always block
  - \* Multiple driver not allowed

17

27 August 2024

## Examples: Reg and Wire

```
module combo(y, a, b);
 input a, b;
 output y;
 wire a, b;
 reg y;

 always @ (a or b)
 begin
 y = a & b;
 end
endmodule
```

```
module seq(y, clk, a);
 input clk, a;
 output y;
 wire clk, a;
 reg y;

 always @ (posedge clk)
 begin
 y = a;
 end
endmodule
```

18

27 August 2024

## Examples: Reg and Wire

```
module combo(y, a, b);
 input a, b;
 output y;
 wire a, b;
 wire y;

 assign y = a & b;

 end
endmodule
```

```
module seq(y, clk, a);
 input clk, a;
 output y;
 wire clk, a;
 reg y;

 always @ (posedge clk)
 begin
 y = a;
 end
endmodule
```

19

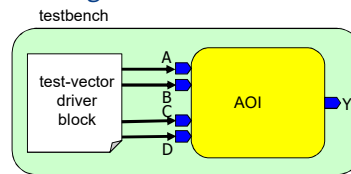
27 August 2024

# Test AOI - testbench

- \* To test AOI
  - \* Drive input ports
  - \* Monitor or check output ports
- \* To drive input ports
  - \* Use columns of truth-table
  - \* Driving value set is known as test-vector
- \* A top-level Verilog module is called testbench
- \* Testbench will instantiate AOI module
- \* Testbench will use a block for driving test-vectors

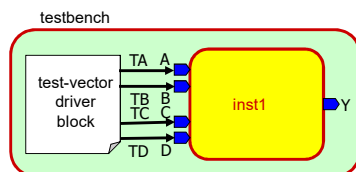


20



27 August 2024

# Verilog testbench



- \* Testbench module
  - \* No ports
  - \* Instantiates AOI

\* Verilog file testbench.v

```
module testbench;
```

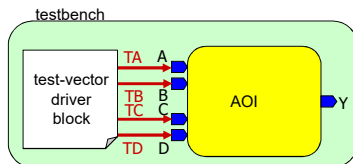
```
// module instantiation
AOI inst1();
```

```
endmodule
```

21

27 August 2024

# Reg declaration for test-vectors



\* Verilog file testbench.v

```
module testbench;
reg TA, TB, TC, TD;
wire TY;
// module instantiation
AOI inst(TY, TA, TB, TC, TD);
```

## \* Testbench module

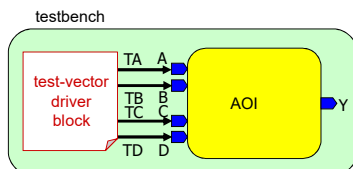
- \* inputs to the AOI are defined with **reg** and the outputs with **wire**

endmodule

22

27 August 2024

# Driver block generating test-vectors



\* Verilog file testbench.v

```
module testbench;
reg TA, TB, TC, TD;
wire TY;
// module instantiation
AOI inst(TY, TA, TB, TC, TD);
initial // drive test vectors
begin
```

## \* Testbench module

- \* The input values are specified with the keyword **initial**
- \* A sequence of values can be specified between **begin** and **end**.

```

 TA = 1'b0; TB = 1'b0;
 TC = 1'b0; TD = 1'b0;
 #10 TD = 1'b1;
 #10 TC = 1'b1; TD = 1'b0;
 #10 TD = 1'b1;
 #10 TB = 1'b1;
 TC = 1'b0; TD = 1'b0;
end
endmodule
```

| A | B | C | D | Y |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 |

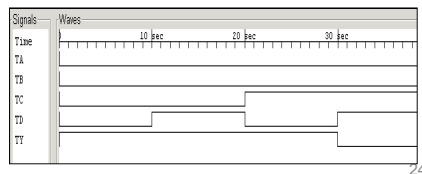
23

27 August 2024

## Monitor block for outputs

- \* Whenever output value changes, monitor should display that

```
0 : 0000 : 1
10: 0001 : 1
20: 0010 : 1
30: 0011 : 0
40: 0100 : 1
```



- \* Verilog file testbench.v

```
module testbench;
reg TA, TB, TC, TD;
wire TY;
// module instantiation
AOI inst(TY, TA, TB, TC, TD);
initial // monitor output
begin
// VCD waveform capture
$dumpvars;
// Log based data monitor
$monitor($time, " : ",
TA, TB, TC, TD, " : ", TY);
// Control simulation time
#100 $finish;
end
endmodule
```

27 August 2024

## Concurrent execution units

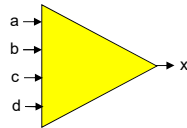
- \* Verilog module can have following concurrent execution units
  - \* Continuous assignment statements – **assign**
  - \* Instantiations (module, gate, udp etc)
  - \* Initial block – **initial**
  - \* Always block – **always**
- \* Such units can execute concurrently – irrespective of where they are declared
- \* Each of such unit represents logic cones driving internal **reg** / **wires** or output / inout ports (ports are by default wire type)
- \* Instantiation and **assign** statements can only drive **wires**
- \* **initial** and **always** blocks can only drive **reg** signals

25

27 August 2024

# Continuous assignment

```
wire x;
assign x = a + b - c * d;
```



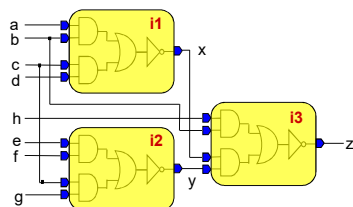
- \* Continuous assignments represents a wire driven by a logic cone i.e. boolean equation.
- \* Executes whenever there is any change in any driving net of the logic cone

26

27 August 2024

# Module instantiation

```
wire x, y, z;
AOI i1(a, b, c, d, x);
AOI i2(e, f, c, g, y);
AOI i3(b, h, x, y, z);
```



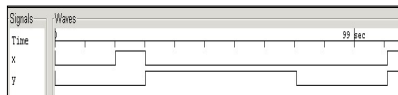
- \* Module instantiation will insert a copy of a parent / instantiating module in the upper module
- \* Only wires can be connected to the output / inout ports of the instances
- \* Each instance of same parent module will represent distinct unit of logic
  - \* There will be no sharing or interference of logic between two instances of same parent module
  - \* Instances are copy of the entire logic of the parent module

27

27 August 2024

# Initial block

```
reg x, y;
initial
begin
 x = 1'b0; y = 1'b0;
 #20 x = 1'b1;
 #10 x = 1'b0; y = 1'b1;
 #50 y = 1'b0;
 #30 x = 1'b1; y = 1'b1;
```



- \* Initial block is mainly used in testbench for generating directed test-vectors
- \* Executes once and starts at the beginning of the simulation
- \* **Can only drive reg signals**
- \* Represents a timed sequence of signal values – waveform
  - \* Timing delays are used to create the timed sequence
  - \* If no timing delay or waits are used, initial block ends instantaneously at zero time

28

27 August 2024

# Examples of initial block

```
* Clock generator
// Clock generator
module clockgen(clock);
parameter period = 10;
output clock;
reg clock;
initial
begin
 clock = 1'b0;
 forever
 #(period/2) clock = ~clock;
end
endmodule
```

- \* Common timing controls used in initial blocks
  - \* Timing delays / waits
    - \* #10;
    - \* wait(10);
    - \* @(x or y);
  - \* System tasks
    - \* \$display
    - \* \$monitor
    - \* \$dumpfile
    - \* \$dumpvars
    - \* \$finish

29

27 August 2024

# System Task

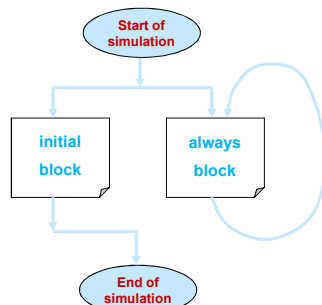
- \* `$display`
  - \* Prints text or variables to stdout.
  - \* Syntax same as “printf”.
- \* `$monitor`
  - \* Similar to `$display`, but prints the value whenever the value of some variable in the given list changes.
- \* `$dumpfile`
  - \* Specify the file that will be used for storing the waveform.
- \* `$dumpvars`
  - \* Starts dumping all the signals to the specified file.
- \* `$finish`
  - \* Terminates the simulation process.

30

27 August 2024

# Always block

- \* Difference between initial and always block executions



- \* Always block is used for the implementation of
  - \* Complex combinational logic
  - \* Sequential logic - flip flop
  - \* Finite state machine (FSM)
- \* Executes repeatedly infinite number of times
- \* **Can only drive reg signals** (why?)
- \* Triggered by any change in signals in the sensitivity list
  - \* `@(x or y or z)`
  - \* `@(posedge clock)`

31

27 August 2024

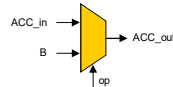
# combinational always

## \* 2x1 Multiplexer



```
always @(s or a or b)
 if (s)
 q = a;
 else
 q = b;
```

## \* ALU



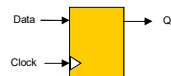
```
always @(ACC_in or B)
 if (op == `ADD)
 ACC_out = ACC_in + B;
 else if (op == `SUB)
 ACC_out = ACC_in - B;
 else if ...
```

32

27 August 2024

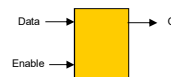
# Sequential always

## \* Flip flop



```
always @(posedge Clock)
 Q = Data;
```

## \* Latch



```
always @(Enable or Data)
 if (enable)
 Q = Data;
```

33

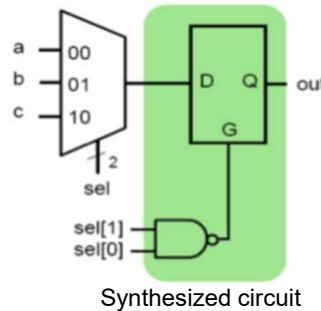
27 August 2024

## Beaware of Incomplete Spec.

- \* In Verilog, if something is not completely specified, the output must retain its previous value when the unspecified condition occurs.

```
module mux_incomplete(out,a,b,c,sel);
input [1:0] sel;
input a,b,c;
output out;
reg out;

always@(a or b or c or sel)
begin
 case (sel)
 2'b00: out = a;
 2'b01: out = b;
 2'b10: out = c;
 endcase
end
endmodule
```



**Solution:** fully specify all branches of if-else construct, or include a default statement in case construct

34

27 August 2024

## Constant Literal

- \* A numeric constant in Verilog can be specified in either the 'sized' or the 'un-sized' format.

- \* <size>'<base><number>

- \* Examples:

- \* 4'b0011 // unsigned 4 bit binary number
- \* 1'b0 // unsigned logic 0
- \* 1'b1 // unsigned logic 1
- \* 20 // signed 32 bit number
- \* 12'hAx0 // unsigned 12'b1010xxxx0000

35

27 August 2024

# Logic Values

- \* The common values used in modeling hardware are
  - \* 0 :: Logic-0 or FALSE
  - \* 1 :: Logic-1 or TRUE
  - \* x :: Unknown (or don't care)
  - \* z :: High impedance
- \* Initialization:
  - \* All unconnected nets set to 'z'
  - \* All register variables set to 'x'

36

27 August 2024

# Primitive Logic Gates

```
* and G (out, in1, in2);
* nand G (out, in1, in2);
* or G (out, in1, in2);
* nor G (out, in1, in2);
* xor G (out, in1, in2);
* xnor G (out, in1, in2);
* not G (out1, in);
* buf G (out1, in);
```

37

27 August 2024

# Representation of vector or bus

- \* Bus or vectors are represented as  

```
reg [3:0] data4;
output [0:7] q;
```
- \* Bus is just collection of a number of signals with same name but different indices
- \* One can perform following vector operations
  - \* Bit wise operation
  - \* Bit select
  - \* Part select
  - \* Concatenation
- \* Bitwise operation  

```
reg [3:0] data;
assign is_all_zero = & data;
```
- \* Bit select  

```
// Multiplexer
assign q = data[i];
```
- \* Part select  

```
// Address latch enable
address[15:8] = address_high;
if (ale) address[7:0] = data;
```
- \* Concatenation  

```
// Bit reversal
reg [3:0] data;
data = { data[0], data[1],
 data[2], data[3] };
```

38

27 August 2024

# Blocking vs Nonblocking Assign.

- Verilog has two different types of assignments: blocking and nonblocking
- Blocking assignments (=) are executed in the order they appear. Therefore the first statement “blocks” the second until it is done, hence it is called blocking assignments

```
a = b;
b = a;
// both a & b = b
```

**blocking**

```
always @ (a or b or c)
begin
 x = a | b;
 y = a ^ b ^ c;
 z = b & ~c;
end
```

**blocking**

1. Evaluate  $a | b$ , assign result to  $x$
2. Evaluate  $a \wedge b \wedge c$ , assign result to  $y$
3. Evaluate  $b \& \sim c$ , assign result to  $z$

- Non-blocking assignments (<=) are executed in parallel. Therefore an earlier statement does not block the later statement

```
a <= b;
b <= a;
// swap a and b
```

**Non-blocking**

```
always @ (a or b or c)
begin
 x <= a | b;
 y <= a ^ b ^ c;
 z <= b & ~c;
end
```

**Non-blocking**

1. Evaluate  $a | b$  but defer assignment of  $x$
2. Evaluate  $a \wedge b \wedge c$  but defer assignment of  $y$
3. Evaluate  $b \& \sim c$  but defer assignment of  $z$
4. Assign  $x$ ,  $y$ , and  $z$  with their new values

39

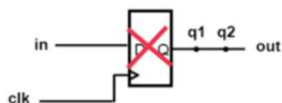
27 August 2024

## Blocking vs Nonblocking Assign.

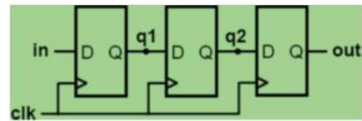
- Consider the two versions of a 3-bit shift register
- Will they give the same results?

```
module blocking(in, clk, out);
 input in, clk;
 output out;
 reg q1, q2, out;
 always @ (posedge clk)
 begin
 q1 = in;
 q2 = q1;
 out = q2;
 end
endmodule
```

- At each rising clock edge:  
q1 = in  
then, q2 = q1 = in  
then, out = q2 = q1 = in
- Therefore, out = in (!!)



**Note:** Use nonblocking assignments for sequential logic



```
module nonblocking(in, clk, out);
 input in, clk;
 output out;
 reg q1, q2, out;
 always @ (posedge clk)
 begin
 q1 <= in;
 q2 <= q1;
 out <= q2;
 end
endmodule
```

- At each rising clock edge, q1, q2, and out simultaneously receive the old values of in, q1, and q2, respectively

## Blocking and Non-blocking Assignments continued

```
always @(posedge clock)
begin
 A <= B;
 C <= A;
end
```

- If always block has no other timing wait except the sensitivity list, you can consider that scheduled non-blocking assignment happens at the end of always block

### \* Design guidelines

- \* Separate combinational and sequential blocks, so that later only represents flip flops
- \* For combinational block, use only blocking assignments
- \* For sequential block, use only non-blocking assignments
- \* Do not mix them. If you mix, you might find:
  - \* Unexplainable behavior
  - \* Simulation output varies with simulation tool

# ALU Design

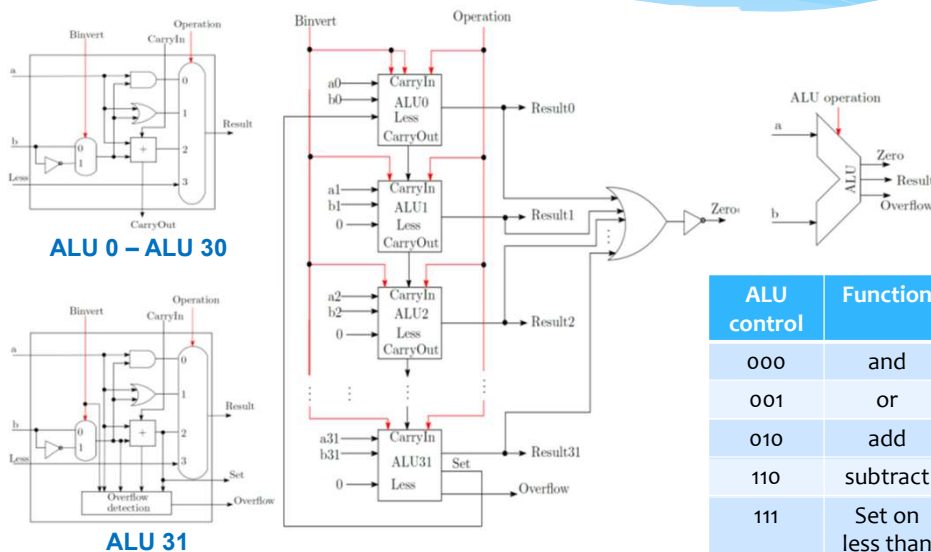
```

module ALU (ALUOut, Zero, ALUctl, A, B);
 input [2:0] ALUctl;
 input [31:0] A,B;
 output reg [31:0] ALUOut;
 output Zero;
 //Zero is true if ALUOut is 0
 assign Zero = (ALUOut==0);
 //reevaluate if these change
 always @(ALUctl, A, B)
 case (ALUctl)
 0: ALUOut <= A & B;
 1: ALUOut <= A | B;
 2: ALUOut <= A + B;
 6: ALUOut <= A - B;
 7: ALUOut <= A < B ? 1:0;
 default: ALUOut <= 0; //default to 0, should not happen;
 endcase
endmodule

```

| ALU control | Function         |
|-------------|------------------|
| 000         | and              |
| 001         | or               |
| 010         | add              |
| 110         | subtract         |
| 111         | Set on less than |

# ALU Design



| ALU control | Function         |
|-------------|------------------|
| 000         | and              |
| 001         | or               |
| 010         | add              |
| 110         | subtract         |
| 111         | Set on less than |

# References

- \* IEEE Standard Verilog® Hardware Description Language (IEEE Std 1364-2001)
- \* IEEE Standard for System Verilog (IEEE Std 1800™-2009)
- \* Understanding Verilog Blocking and Non-blocking Assignments - International Cadence User Group Conference, September 11, 1996 presented by Stuart Sutherland